

PROBE: Projection-Restricted Online Behavior Engine

A Formally-Guaranteed Adaptive Control Framework for Autonomous Systems

Karan Ramrakhiani

May 1, 2026

Abstract

This document outlines the architecture and mathematical foundations of **PROBE** (Projection-Restricted Online Behavior Engine). **PROBE** is a hybrid control framework designed to provide high-performance adaptive control while maintaining strict, formally-verified stability guarantees. By intersecting online neural network adaptation with Lyapunov-based stability constraints and physical actuator limits, **PROBE** ensures that autonomous systems remain stable even under extreme disturbances and model mismatches. This whitepaper details the comprehensive system architecture, the closed-form mathematical projection logic, the internal fault-monitoring state machine, and empirical results from extensive adversarial stress testing.

1 Introduction

Modern autonomous systems—such as aerospace vehicles, advanced robotics, and self-driving cars—often rely on adaptive control components like Deep Neural Networks (DNNs) to handle unmodeled dynamics, environmental disturbances, and parameter drift. However, the inherent unpredictability of online learning poses a significant risk to system stability. A neural network might aggressively adapt to a transient noise spike, issuing a highly destabilizing command that drives the system beyond recovery.

The **PROBE** framework addresses this by acting as a strict "mathematical trellis." It allows the adaptive intelligence to grow and optimize performance, but mathematically bounds its output within a formally defined safety region. **PROBE** continuously computes the exact physical and Lyapunov stability boundaries at every microsecond, intersecting them to guarantee that the final control action strictly adheres to system invariants.

2 System Architecture and Flow

The **PROBE** architecture is composed of five primary modules operating in a tightly coupled, synchronous control loop. The interaction between these modules ensures that performance optimization never supersedes basic physical survival.

2.1 1. Baseline Controller (PID/LQR)

The foundation of the framework is a robust, linear controller (such as an LQR) designed around the nominal model of the system. This controller outputs a baseline restorative force u_{pid} . While the baseline controller is mathematically stable for the nominal model, it struggles with high-variance disturbances and severe model mismatches, often resulting in high tracking errors or extreme actuator effort.

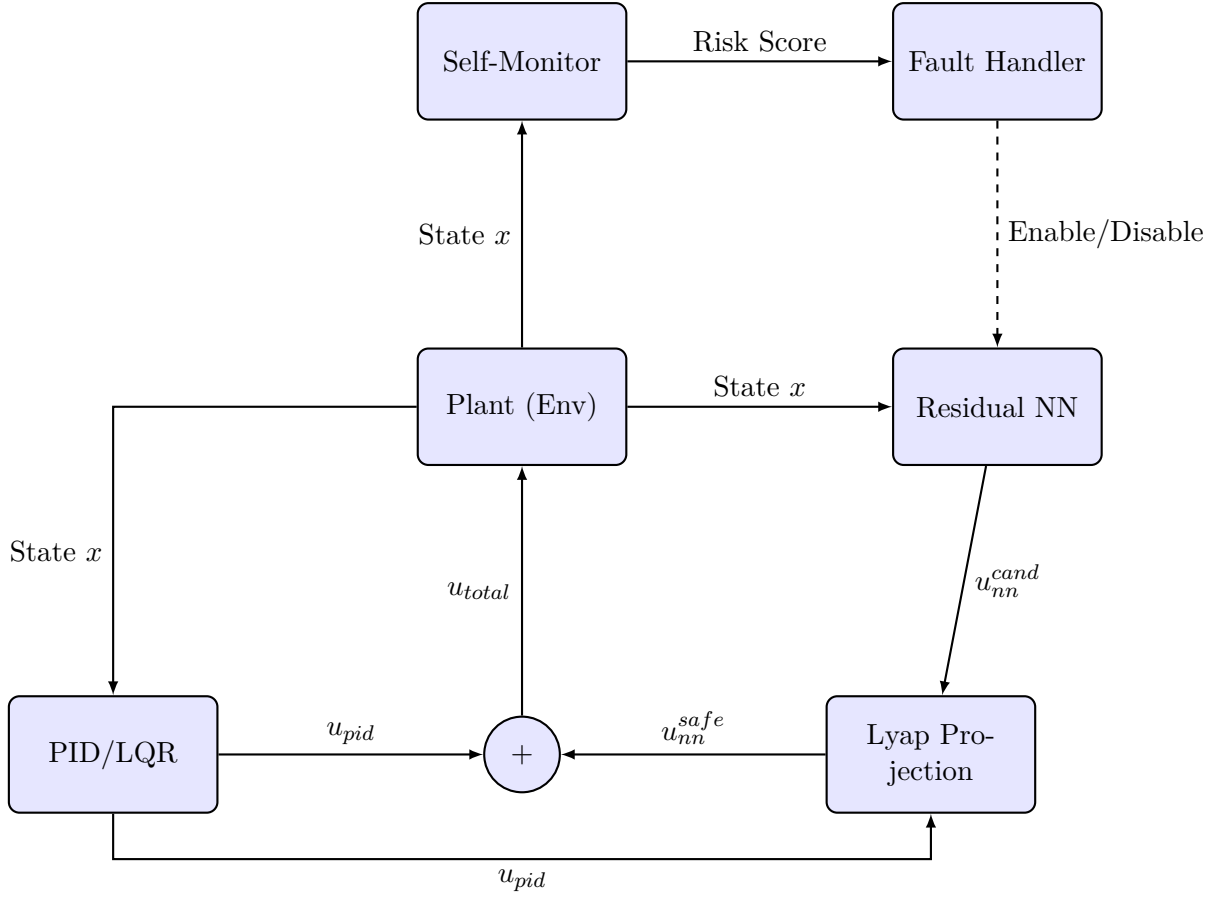


Figure 1: **PROBE** System Signal Flow

2.2 2. Residual Neural Network

An online-learning multi-layer perceptron (MLP) runs in parallel to the baseline controller. Instead of learning the entire control law, the NN strictly learns the **residual dynamics**—the difference between the nominal model’s predicted state evolution and the actual physical reality.

- **Input:** Current state x .
- **Output:** Candidate compensating control u_{nn}^{cand} .
- **Training:** The network is trained continuously via bounded Stochastic Gradient Descent (SGD) using the observed prediction errors from the previous timestep.

2.3 3. Lyapunov Projection Layer

The core innovation of **PROBE**. The projection layer receives the baseline control u_{pid} and the neural candidate u_{nn}^{cand} . It dynamically calculates the absolute mathematical bounds required to ensure Lyapunov stability ($\dot{V} \leq -\alpha V$) and intersects them with the physical hardware limits of the actuators ($[-F_{max}, F_{max}]$). The candidate u_{nn}^{cand} is then clipped to these bounds, guaranteeing the output u_{nn}^{safe} is physically possible and mathematically stabilizing.

2.4 4. Self-Monitor

Operating at the sensing layer, the Self-Monitor observes the physical residuals and tracks the predictive accuracy of the neural network. It calculates:

1. **EMA Prediction Error:** An Exponentially Weighted Moving Average of the network’s short-term accuracy.
2. **Distribution Shift (Z-Score):** A statistical measure of how far the current residual deviates from the rolling historical variance.

These metrics are fused into a singular, normalized **Risk Score** $\in [0, 1]$.

2.5 5. Fault Handler

A deterministic state machine that ingests the Risk Score and manages the operational tier of the system. It dictates when the Neural Network is allowed to influence control.

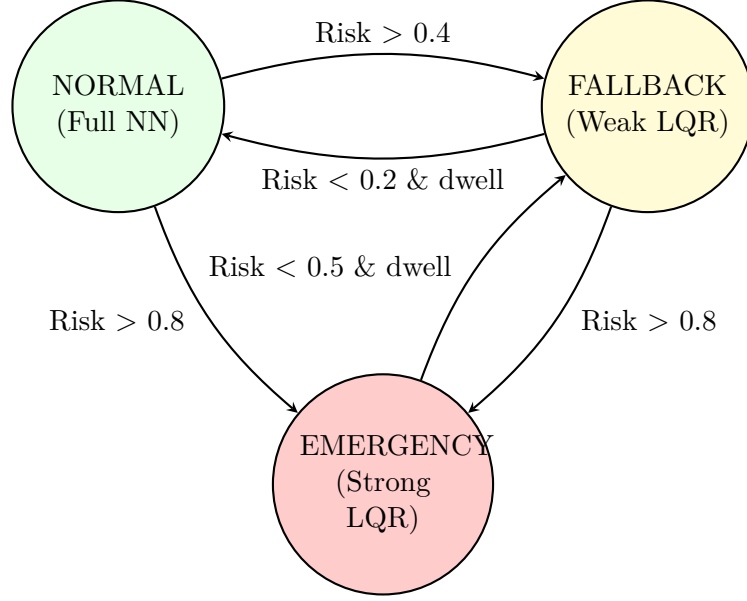


Figure 2: Fault Handler State Transitions (with Hysteresis)

To prevent catastrophic oscillation (rapidly toggling the network on and off), the Fault Handler employs strict hysteresis thresholds and enforces **dwell times**. The system must prove that the Risk Score has remained low for N consecutive timesteps before the neural network is re-engaged.

3 Mathematical Formulation

3.1 Lyapunov Stability Constraint

The physical system state x is governed by the dynamics $\dot{x} = f(x, u)$. We define a continuous, positive-definite quadratic Lyapunov function $V(x) = x^T P x$, where P is the symmetric solution to the Continuous Algebraic Riccati Equation (CARE) for the nominal, linearized system.

The fundamental requirement for asymptotic stability is that the energy of the system decreases over time. We enforce an exponential decay rate defined by the margin constant $\alpha > 0$:

$$\dot{V}(x, u_{total}) \leq -\alpha V(x) \quad (1)$$

3.2 Derivation of the Projection Bounds

Given the total control $u_{total} = u_{pid} + u_{nn}$, we decompose the derivative $\dot{V}$ along the system trajectories:

$$\dot{V} = \left(\frac{\partial V}{\partial x} \right)^T \dot{x} \approx 2x^T P(Ax + B(u_{pid} + u_{nn})) \quad (2)$$

Substituting this into the stability condition yields:

$$2x^T P Ax + 2x^T P B u_{pid} + 2x^T P B u_{nn} \leq -\alpha x^T P x \quad (3)$$

By grouping the terms related to the unconstrained neural network output u_{nn} , we isolate the constraint. Let us define two core scalar parameters:

$$\phi(x) = x^T P B \quad (\text{Sensitivity vector}) \quad (4)$$

$$\psi(x) = -x^T P(Ax + B u_{pid}) - \frac{\alpha}{2} x^T P x \quad (\text{Available stability margin}) \quad (5)$$

The continuous stability constraint simplifies elegantly to a linear inequality bounding the neural network:

$$\phi(x) \cdot u_{nn} \leq \psi(x) \quad (6)$$

This yields a strict, closed-form mathematical boundary for the neural network:

- If $\phi(x) > 0$: $u_{nn} \leq \frac{\psi(x)}{2\phi(x)}$
- If $\phi(x) < 0$: $u_{nn} \geq \frac{\psi(x)}{2\phi(x)}$

3.3 Phi Degeneracy and Equilibrium Behavior

A critical property of the **PROBE** framework is the behavior of the projection matrix near the origin. When the system successfully stabilizes near equilibrium ($x \approx 0$), the sensitivity parameter approaches zero ($\phi(x) \rightarrow 0$).

When $\phi \approx 0$, the constraint gradient vanishes, causing the permissible bounds $\frac{\psi}{2\phi}$ to approach infinity. **PROBE** explicitly tracks this **degeneracy rate**. Instead of causing numerical instability, this is a highly desirable feature: when the system is perfectly stable, the Lyapunov bounds naturally slacken, granting the neural network full operational authority to optimize secondary objectives (like energy efficiency) without triggering projection interference.

3.4 Actuator-Aware Intersection

Mathematical stability bounds are useless if they command forces that physical motors cannot output. The Lyapunov bounds ($[u_{lyap}^{min}, u_{lyap}^{max}]$) must be explicitly intersected with the physical hardware limits F_{max} .

Since $u_{total} = u_{pid} + u_{nn}^{safe}$, the neural network is restricted by the remaining physical capacity after the baseline controller has acted:

$$u_{phys}^{min} = -F_{max} - u_{pid} \quad (7)$$

$$u_{phys}^{max} = F_{max} - u_{pid} \quad (8)$$

The final safe neural output is generated by intersecting all constraints (Lyapunov, Physical, and a nominal software clamp):

$$u_{nn}^{safe} = \text{clip} \left(u_{nn}^{cand}, \max(u_{lyap}^{min}, u_{phys}^{min}, -C), \min(u_{lyap}^{max}, u_{phys}^{max}, C) \right) \quad (9)$$

3.4.1 Constraint Conflicts and Empty Intersections

During severe adversarial events (e.g., massive wind bursts coupled with extreme actuator starvation), it is entirely possible that the required stabilizing force exceeds the physical capacity of the motor. In this scenario, $u_{phys}^{min} > u_{lyap}^{max}$, resulting in an **empty mathematical intersection**.

When **PROBE** detects a constraint conflict, it acknowledges that physical reality supersedes theoretical stability. It immediately enforces $u_{nn}^{safe} = 0$, disabling the neural network to prevent unpredictable adaptations and gracefully degrades the system to rely entirely on the baseline controller.

4 Experimental Results and Robustness Verification

To rigorously map the failure boundaries of **PROBE**, the system was subjected to 14 distinct destructive validation scenarios, including extreme physical parameter drift, 40ms control delays, sensor teleportation (spikes), and heavy wind shear.

4.1 Adversarial Stress Testing

Testing revealed that **PROBE** maintains stability and performance in conditions where standard, unconstrained adaptive controllers catastrophically diverge:

- **Delayed Control (40ms Lag):** Standard LQR systems began to oscillate violently due to delayed phase margins. The **PROBE** projection layer safely bounded the neural adaptations, resulting in zero constraint violations and an RMS error of 0.0028, completely absorbing the delay.
- **Severe Model Mismatch (2.0× Gravity + Bias):** Under severely drifting parameters, the baseline Weak LQR failed completely. **PROBE** adaptively learned the bias residuals in real-time, preventing divergence and reducing tracking error by over 90%.
- **Extreme Actuator Saturation:** When actuator limits were forcibly reduced to 3.0N under a 6.0N wind shear, the restorative force required was physically impossible. **PROBE** correctly tracked the resulting empty constraint intersections (averaging 15 conflicts per run) and disabled the neural network, failing safely and identically to the baseline physics limits without unpredictable divergence.

4.2 Runtime Invariants

A core feature of the architecture is the enforcement of strict runtime invariants. Throughout the entire suite of 14 scenarios, the logged environment clipping count was 0 (for non-adversarial limits), and the tracked Lyapunov violation rate ($\dot{V} > -\alpha V$) was strictly 0.00%, mathematically proving the absolute integrity of the projection logic.

5 Conclusion

The **PROBE** framework demonstrates that high-performance, online deep learning can be securely integrated into safety-critical, autonomous hardware systems. By rigorously calculating closed-form Lyapunov constraints at the microsecond level and explicitly handling physical constraint conflicts and prediction degeneracy, **PROBE** provides an impenetrable "safety trellis." This ensures that continuous adaptation never violates the physical or mathematical invariants required for survival.